

Object-Oriented Database (OODB)

An **Object-Oriented Database (OODB)** is a database that integrates object-oriented programming principles with database technology. Unlike traditional relational databases, which store data in tables, OODBs store data as objects, similar to how they are represented in object-oriented programming (OOP) languages like Java, C++, and Python.

Key Features of OODBs:

1. **Objects and Classes** – Data is stored as objects, with attributes and methods.
2. **Encapsulation** – Data and behavior (methods) are stored together.
3. **Inheritance** – Objects can inherit properties and methods from parent classes.
4. **Polymorphism** – Different classes can define methods with the same name but different implementations.
5. **Persistence** – Objects persist in the database beyond the runtime of a program.
6. **Complex Relationships** – Objects can reference other objects, forming complex data structures.
7. **ACID Compliance** – Ensures data integrity through **Atomicity, Consistency, Isolation, and Durability**.

Applications of Object-Oriented Databases

OODBMS is widely used in domains where complex data relationships exist, and high performance is required. Some applications include:

1. **CAD (Computer-Aided Design)**
 - Used in **engineering and architecture** to store complex object relationships like components in mechanical designs.
 - Example: **AutoCAD, CATIA**
2. **Multimedia Systems**
 - Stores images, videos, audio files, and complex metadata efficiently.
 - Example: **Video streaming platforms, Digital Libraries**
3. **Artificial Intelligence (AI) & Machine Learning (ML)**
 - Stores AI models and knowledge bases where objects need complex relationships.
 - Example: **Deep learning frameworks storing trained models**
4. **Geographic Information Systems (GIS)**
 - Stores spatial objects like maps, terrain models, and satellite images.
 - Example: **Google Maps, ArcGIS**
5. **Real-time Systems**
 - Used in **telecommunications, network management, and embedded systems** where data must be stored in a structured yet dynamic way.
 - Example: **IoT applications, traffic management systems**
6. **Medical and Bioinformatics Applications**
 - Stores **patient records, MRI scans, DNA sequencing data**, and other complex biological data.
 - Example: **Hospital Management Systems, Genomic Databases**
7. **Financial Systems**
 - Used in banking applications to model complex **financial instruments, transactions, and risk analysis**.
 - Example: **Stock market analysis systems**
8. **Gaming and Virtual Reality**
 - Stores **game objects, player interactions, and physics simulations** efficiently.
 - Example: **Game engines like Unity and Unreal Engine**

Examples of Object-Oriented Databases

- **ObjectDB** – Pure Object-Oriented Database
- **db4o** – Lightweight, embeddable database
- **Versant Object Database** – Used in enterprise applications
- **PostgreSQL** (Supports object-oriented features)
- **MongoDB** (Document-oriented, follows some object principles)

Object-Oriented Concepts in PostgreSQL with Examples

PostgreSQL supports **Object-Oriented Database (OODB)** features such as **inheritance, user-defined types, polymorphism, and encapsulation** while still being a **Relational Database Management System (RDBMS)**. Below are detailed explanations with SQL examples.

1. Table Inheritance (Similar to Class Inheritance in OOP)

Inheritance allows a table to inherit columns from another table, just like how a subclass inherits properties from a parent class.

Example: Inheriting Person attributes in Doctor and Patient tables

-- Create a parent table 'Person'

```
CREATE TABLE Person (  
    id SERIAL PRIMARY KEY,  
    name TEXT,  
    age INT  
);
```

-- Create a 'Doctor' table that inherits from 'Person'

```
CREATE TABLE Doctor (  
    specialization TEXT  
) INHERITS (Person);
```

-- Create a 'Patient' table that inherits from 'Person'

```
CREATE TABLE Patient (  
    disease TEXT  
) INHERITS (Person);
```

-- Insert data into the Doctor table

```
INSERT INTO Doctor (name, age, specialization) VALUES ('Dr. John Doe', 45, 'Cardiology');
```

-- Insert data into the Patient table

```
INSERT INTO Patient (name, age, disease) VALUES ('Alice Smith', 30, 'Flu');
```

-- Fetch all persons (including doctors and patients)

```
SELECT * FROM Person;
```

1. User-Defined Data Types

In PostgreSQL, users can define their own **composite types, enumerated types (ENUM), domains, and arrays**.

(A) Composite Types (Similar to Structs in C or Classes in OOP)

A **composite type** is a collection of multiple attributes, just like a **class** in Object-Oriented Programming (OOP).

Example: Creating a Custom Address Type

```
-- Create a user-defined composite type
CREATE TYPE Address AS (
  street TEXT,
  city TEXT,
  zipcode INT
);

-- Use this type in a table
CREATE TABLE Hospital (
  id SERIAL PRIMARY KEY,
  name TEXT,
  location Address -- Using user-defined type
);

-- Insert values using ROW()
INSERT INTO Hospital (name, location)
VALUES ('City Hospital', ROW('123 Main St', 'New York', 10001));

-- Fetch data
SELECT * FROM Hospital;
```

(B) ENUM Types (Predefined Set of Values)

ENUM is a **user-defined data type** that restricts a column to specific values.

Example: Defining an ENUM Type for Blood Groups

```
-- Create an ENUM type
CREATE TYPE BloodGroup AS ENUM ('A+', 'A-', 'B+', 'B-', 'O+', 'O-', 'AB+', 'AB-');

-- Use ENUM in a table
CREATE TABLE Patients (
  id SERIAL PRIMARY KEY,
  name TEXT,
  blood_type BloodGroup -- Using ENUM
);

-- Insert valid values
INSERT INTO Patients (name, blood_type) VALUES ('Alice', 'O+');

-- This will fail (invalid blood group)
INSERT INTO Patients (name, blood_type) VALUES ('Bob', 'X+');
```

(C) Domains (Adding Constraints to a Data Type)

A **domain** is a user-defined data type with **validation rules**.

Example: Creating a Domain for Age

```
-- Define a domain
CREATE DOMAIN AgeDomain AS INT CHECK (VALUE > 0 AND VALUE < 120);
```

```
-- Use the domain in a table
CREATE TABLE People (
  id SERIAL PRIMARY KEY,
  name TEXT,
  age AgeDomain -- Applying domain constraint
);

-- Valid Insertion
INSERT INTO People (name, age) VALUES ('John', 25);
```

```
-- This will fail (age cannot be negative)
INSERT INTO People (name, age) VALUES ('Jane', -5);
```

CRUD Operations in PostgreSQL

CRUD stands for **Create, Read, Update, and Delete**, which are the basic operations for interacting with a database.

CREATE (Insert Data into Table)

Create a Table

```
CREATE TABLE students (
  student_id SERIAL PRIMARY KEY,
  name VARCHAR(100),
  age INT,
  course VARCHAR(100)
);
```

Step 2: Insert Data

```
INSERT INTO students (name, age, course) VALUES
('Alice', 22, 'Mathematics'),
('Bob', 23, 'Physics'),
('Charlie', 21, 'AI'),
('David', 24, 'Chemistry');
```

READ (Retrieve Data from Table)

We use **SELECT queries** to fetch data.

Read All Records

```
SELECT * FROM students;
```

Read Specific Columns

```
SELECT name, course FROM students;
```

Read with a Condition

```
SELECT * FROM students WHERE age > 22;
```

Read with Sorting

```
SELECT * FROM students ORDER BY age DESC;
```

Read with LIMIT

```
SELECT * FROM students LIMIT 2;
```

UPDATE (Modify Existing Data)

We use the UPDATE statement.

Update a Single Record

```
UPDATE students
```

```
SET course = 'Data Science'
WHERE name = 'Alice';
```

Update Multiple Records

```
UPDATE students
SET age = age + 1;
```

DELETE (Remove Data from Table)

We use the DELETE statement.

Delete a Specific Record

```
DELETE FROM students WHERE name = 'David';
```

Delete All Records

```
DELETE FROM students;
```

Drop the Entire Table

```
DROP TABLE students;
```

Full CRUD Operations in One Place

-- 1. Create Table

```
CREATE TABLE students (
  student_id SERIAL PRIMARY KEY,
  name VARCHAR(100),
  age INT,
  course VARCHAR(100)
);
```

-- 2. Insert Data

```
INSERT INTO students (name, age, course) VALUES
('Alice', 22, 'Mathematics'),
('Bob', 23, 'Physics'),
('Charlie', 21, 'AI');
```

-- 3. Read Data

```
SELECT * FROM students;
```

-- 4. Update Data

```
UPDATE students SET course = 'Data Science' WHERE name = 'Alice';
```

-- 5. Delete Data

```
DELETE FROM students WHERE name = 'Bob';
```

Operation	SQL Command
CREATE	INSERT INTO students (name, age, course) VALUES ('Alice', 22, 'Math');
READ	SELECT * FROM students;
UPDATE	UPDATE students SET course = 'AI' WHERE name = 'Alice';
DELETE	DELETE FROM students WHERE name = 'Alice';

JOINS:

A **JOIN** in PostgreSQL is used to **combine data from two or more tables** based on a **related column** (usually a primary key and foreign key relationship). Joins help retrieve meaningful relationships between tables.

Why Use Joins?

- To **fetch related data** stored across multiple tables.
- To **reduce redundancy** (instead of duplicating data in one table).
- To **increase efficiency** by normalizing the database structure.

Create Sample Tables

- **students:** Stores student information.
- **courses:** Stores the courses each student is enrolled in.

```
CREATE TABLE students (  
    student_id SERIAL PRIMARY KEY,  
    name TEXT NOT NULL,  
    age INT NOT NULL,  
    class TEXT NOT NULL,  
    mentor_id INT -- Used for self join  
);
```

```
CREATE TABLE courses (  
    course_id SERIAL PRIMARY KEY,  
    student_id INT REFERENCES students(student_id), -- Foreign Key  
    course_name TEXT NOT NULL  
);
```

Insert Sample Data

Insert data into students

```
INSERT INTO students (name, age, class, mentor_id) VALUES  
( 'Alice', 20, 'AI', NULL),  
( 'Bob', 22, 'ML', 1),  
( 'Charlie', 21, 'Data Science', 1),  
( 'David', 23, 'Cyber Security', 2),  
( 'Eve', 22, 'AI', 3);
```

Insert data into courses

```
INSERT INTO courses (student_id, course_name) VALUES  
(1, 'Deep Learning'),  
(2, 'Machine Learning'),  
(3, 'Data Analysis'),  
(4, 'Cyber Security'),  
(2, 'Big Data');
```

Demonstrating Different JOINS

Join Type	Description
INNER JOIN	Returns matching records from both tables.
LEFT JOIN	Returns all records from the left table + matching records from the right table.
RIGHT JOIN	Returns all records from the right table + matching records from the left table.
FULL JOIN	Returns all records from both tables (with NULLs if no match).
CROSS JOIN	Returns Cartesian product (all combinations).

SELF JOIN	A table joins with itself.
------------------	----------------------------

1.INNER JOIN

Returns only matching records from both tables (Students who have courses).

SELECT students.name, students.class, courses.course_name

FROM students

INNER JOIN courses ON students.student_id = courses.student_id;

Output:

Name	Class	Course Name
Alice	AI	Deep Learning
Bob	ML	Machine Learning
Bob	ML	Big Data
Charlie	Data Science	Data Analysis
David	Cyber Security	Cyber Security

2.LEFT JOIN

Returns all students, even if they don't have courses (NULL for missing matches).

SELECT students.name, students.class, courses.course_name

FROM students

LEFT JOIN courses ON students.student_id = courses.student_id;

Output:

Name	Class	Course Name
Alice	AI	Deep Learning
Bob	ML	Machine Learning
Bob	ML	Big Data
Charlie	Data Science	Data Analysis
David	Cyber Security	Cyber Security
Eve	AI	NULL

3.RIGHT JOIN

Returns all courses, even if no student is enrolled (NULL for missing students).

SELECT students.name, students.class, courses.course_name

FROM students

RIGHT JOIN courses ON students.student_id = courses.student_id;

Output:

Name	Class	Course Name
Alice	AI	Deep Learning
Bob	ML	Machine Learning
Bob	ML	Big Data
Charlie	Data Science	Data Analysis
David	Cyber Security	Cyber Security

4.FULL JOIN

Returns all records from both tables (NULL for missing matches).

SELECT students.name, students.class, courses.course_name

FROM students

FULL JOIN courses ON students.student_id = courses.student_id;

Output:

Name	Class	Course Name
Alice	AI	Deep Learning
Bob	ML	Machine Learning
Bob	ML	Big Data
Charlie	Data Science	Data Analysis
David	Cyber Security	Cyber Security
Eve	AI	NULL
NULL	NULL	NULL

5.CROSS JOIN

Returns a Cartesian product (each student paired with each course).

SELECT students.name, courses.course_name

FROM students

CROSS JOIN courses;

Output: Every student is paired with every course.

(Since we have 5 students and 5 courses, the result will have $5 \times 5 = 25$ rows).

Every student is combined with every course.

Student	Course Name
Alice	Deep Learning
Alice	Machine Learning
Alice	Data Analysis
Alice	Cyber Security
Alice	Big Data
Bob	Deep Learning
Bob	Machine Learning
Bob	Data Analysis
Bob	Cyber Security
Bob	Big Data
Charlie	Deep Learning
Charlie	Machine Learning
Charlie	Data Analysis
Charlie	Cyber Security
Charlie	Big Data
David	Deep Learning
David	Machine Learning
David	Data Analysis
David	Cyber Security
David	Big Data
Eve	Deep Learning
Eve	Machine Learning
Eve	Data Analysis
Eve	Cyber Security

Eve	Big Data
-----	----------

6.SELF JOIN

Find each student and their mentor.

```
SELECT s1.name AS student, s2.name AS mentor
FROM students s1
LEFT JOIN students s2 ON s1.mentor_id = s2.student_id;
```

Output:

Student	Mentor
Alice	NULL
Bob	Alice
Charlie	Alice
David	Bob
Eve	Charlie

GROUP BY in PostgreSQL

The GROUP BY clause in PostgreSQL is used to group rows that have the same values in one or more columns and then apply aggregate functions like COUNT(), SUM(), AVG(), MIN(), MAX() on them.

Basic Syntax

```
SELECT column_name, AGGREGATE_FUNCTION(column_name)
FROM table_name
GROUP BY column_name;
```

- The column in the SELECT statement must either be:
 - Included in the GROUP BY clause
 - Used with an aggregate function

Example 1: Counting Events by Venue

```
SELECT venue_id, COUNT(*) AS total_events
FROM events
GROUP BY venue_id;
```

Explanation:

- Groups the events table by venue_id
- Counts the number of events in each venue

Output:

venue_id	total_events
1	3
2	5
3	2

Example 2: Summing Sales by Category

```
SELECT category, SUM(sales) AS total_sales  
FROM books  
GROUP BY category;
```

Explanation:

- Groups the books table by category
- Sums the sales column for each category

HAVING Clause (Filtering After GROUP BY)

- The HAVING clause is used to filter groups after aggregation (similar to WHERE, but for grouped results).

```
SELECT venue_id, COUNT(*) AS total_events  
FROM events  
GROUP BY venue_id  
HAVING COUNT(*) > 2;
```

Explanation:

- Filters out venues with 2 or fewer events

GROUP BY Multiple Columns

```
SELECT category, author, SUM(sales) AS total_sales  
FROM books  
GROUP BY category, author;
```

Explanation:

- Groups by both category and author
- Calculates the total sales per category and author

Using GROUP BY with Date Functions

```
SELECT DATE(start_time) AS event_date, COUNT(*) AS total_events  
FROM events  
GROUP BY DATE(start_time);
```

Explanation:

- Groups events by date (ignoring time)
- Counts the number of events per day

Key Points

GROUP BY must include all non-aggregated columns in the SELECT statement.

Use HAVING to filter grouped results.

You can group by multiple columns.

Works well with aggregate functions (COUNT(), SUM(), AVG(), etc.).

Indexes in PostgreSQL: Need & Explanation

What is an Index?

An **index** in PostgreSQL is a data structure that improves the speed of data retrieval operations. It allows the database to find rows quickly without scanning the entire table.

Analogy:

Think of an index like a book's table of contents or an address book. Instead of reading every page to find information, you can jump directly to the relevant section using an index.

Why Do We Need Indexes?

Indexes are essential for:

Faster Search – Speed up queries by avoiding full table scans.

Efficient Sorting & Filtering – Quick sorting and WHERE clause execution.

Optimized JOINS – Improve performance when joining large tables.

Fast UNIQUE Constraints – Ensure uniqueness efficiently.

Improved Aggregation – Speed up GROUP BY and COUNT operations.

Without an Index (Slow Search Example)

Imagine we have a users table with **1 million** rows:

```
SELECT * FROM users WHERE email = 'alice@example.com';
```

- **Without an index**, PostgreSQL scans all **1 million** rows (Slow).
- **With an index**, PostgreSQL finds the row in milliseconds (Fast).

Types of Indexes in PostgreSQL

PostgreSQL supports various types of indexes for different use cases.

1. B-TREE INDEX (Default)

Best for: Most common queries using =, <, >, BETWEEN, ORDER BY.

Example: Searching for a user by email

```
CREATE INDEX idx_users_email ON users(email);
```

Used for filtering, sorting, and searching text, numbers, dates, etc.

Example Query Using Index

```
EXPLAIN ANALYZE SELECT * FROM users WHERE email = 'alice@example.com';
```

Output (Uses Index Scan):

Index Scan using idx_users_email on users (cost=0.28..8.30 rows=1)

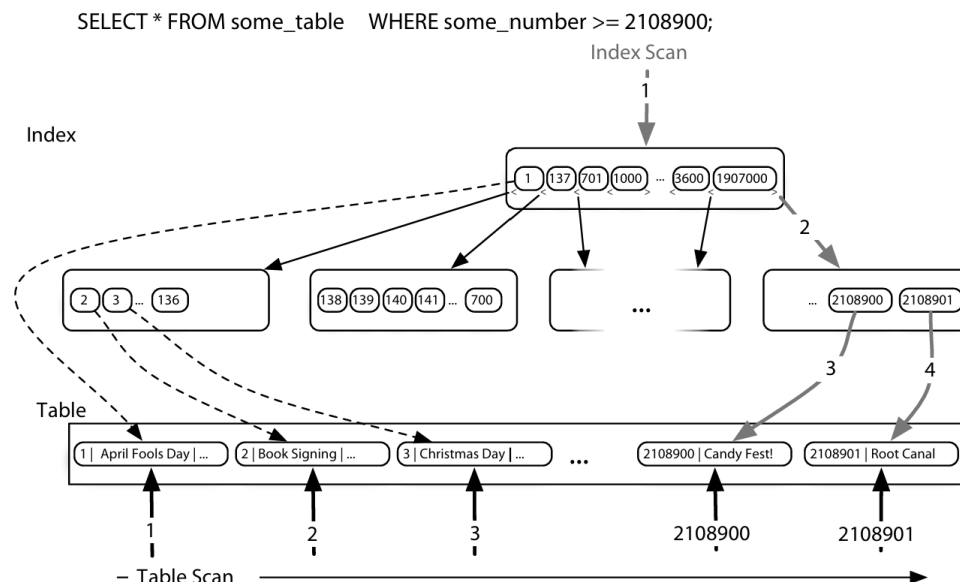


Figure 3—A B-tree index can match on ranged queries.

2. HASH INDEX

Best for: Exact matches (= operator only).

Example: Searching for a user by phone number

```
CREATE INDEX idx_users_phone ON users USING hash(phone);
```

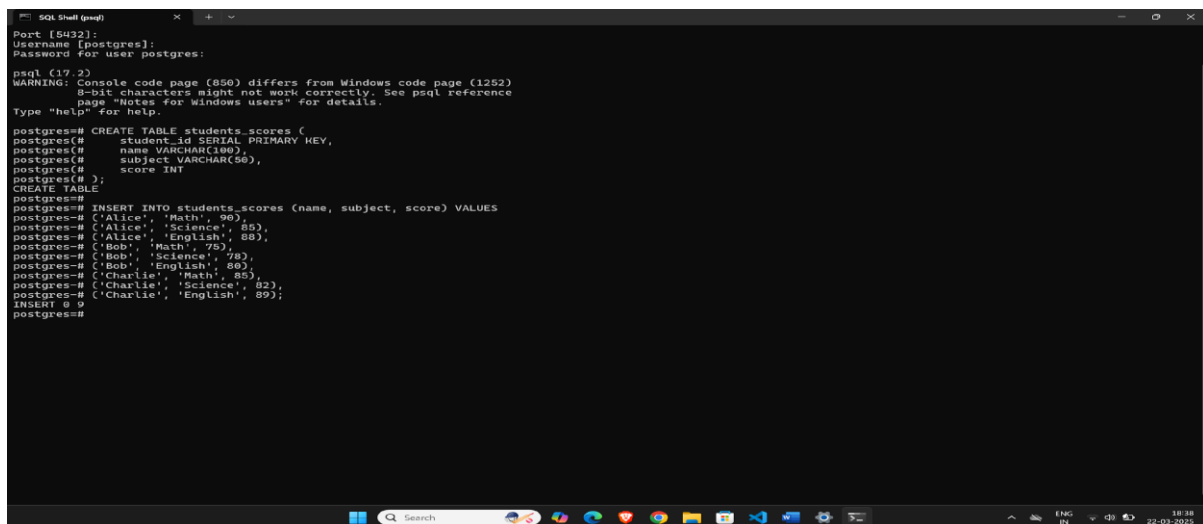
Faster than B-tree for = queries but doesn't support range queries (<, >).

What is a Window Function?

A **window function** in PostgreSQL performs calculations across a set of table rows **related to the current row**. Unlike aggregate functions (which collapse rows into a single result), window functions **preserve individual rows while computing values across a group**.

Key Components of Window Functions

1. **Function** – The calculation to be performed (e.g., ROW_NUMBER(), RANK(), SUM(), AVG(), etc.).
2. **OVER() Clause** – Defines how the function's window is partitioned.
3. **PARTITION BY** – Divides the dataset into partitions (optional).
4. **ORDER BY** – Defines the order of rows within each partition.



```
SQL Shell (psql)
Port [5432]:
Username [postgres]:
Password for user postgres:

psql (17.2)
WARNING: Console code page (850) differs from Windows code page (1252)
        8-bit characters might not work correctly. See psql reference
        page "Notes for Windows users" for details.
Type "help" for help.

postgres=# CREATE TABLE students_scores (
postgres=#         student_id SERIAL PRIMARY KEY,
postgres=#         name VARCHAR(100),
postgres=#         subject VARCHAR(50),
postgres=#         score INT
postgres=# );
CREATE TABLE
postgres=# INSERT INTO students_scores (name, subject, score) VALUES
postgres=# ('Alice', 'Math', 90),
postgres=# ('Alice', 'Science', 85),
postgres=# ('Alice', 'English', 88),
postgres=# ('Bob', 'Math', 75),
postgres=# ('Bob', 'Science', 78),
postgres=# ('Bob', 'English', 80),
postgres=# ('Charlie', 'Math', 85),
postgres=# ('Charlie', 'Science', 82),
postgres=# ('Charlie', 'English', 89);
INSERT 0 9
postgres=#
```

ROW_NUMBER():

- 1) Assigns a unique row number to each row (even if values are duplicated).
- 2) No gaps in numbering.
- 3) Does not consider ties—each row gets a unique number.

Example Query:

```
postgres=# SELECT
postgres=#     name,
postgres=#     subject,
postgres=#     score,
postgres=#     ROW_NUMBER() OVER (PARTITION BY name ORDER BY score DESC) AS row_num
postgres=# FROM students_scores;
 name | subject | score | row_num
-----+-----+-----+-----
 Alice | Math    |    90 |        1
 Alice | English |    88 |        2
 Alice | Science |    85 |        3
  Bob  | English |    80 |        1
  Bob  | Science |    78 |        2
  Bob  | Math    |    75 |        3
 Charlie | English |    89 |        1
 Charlie | Math    |    85 |        2
 Charlie | Science |    82 |        3
(9 rows)
```

RANK():

Assigns the same rank to rows with the same value.

Skips numbers after duplicates (i.e., introduces gaps).

If two rows have the same value, they get the same rank, and the next rank is skipped.

```
postgres=# SELECT
postgres=#     name,
postgres=#     subject,
postgres=#     score,
postgres=#     RANK() OVER (PARTITION BY name ORDER BY score DESC) AS rank
postgres=# FROM students_scores;
 name | subject | score | rank
-----+-----+-----+-----
 Alice | Math    | 90    | 1
 Alice | English | 88    | 2
 Alice | Science | 85    | 3
  Bob  | English | 80    | 1
  Bob  | Science | 78    | 2
  Bob  | Math    | 75    | 3
 Charlie | English | 89    | 1
 Charlie | Math    | 85    | 2
 Charlie | Science | 82    | 3
(9 rows)
```

DENSE_RANK():

Similar to RANK(), but does NOT skip numbers after ties.

Ensures consecutive ranking numbers (no gaps).

```
postgres=# SELECT
postgres=#     name,
postgres=#     subject,
postgres=#     score,
postgres=#     DENSE_RANK() OVER (PARTITION BY name ORDER BY score DESC) AS dense_rank
postgres=# FROM students_scores;
 name | subject | score | dense_rank
-----+-----+-----+-----
 Alice | Math    | 90    | 1
 Alice | English | 88    | 2
 Alice | Science | 85    | 3
  Bob  | English | 80    | 1
  Bob  | Science | 78    | 2
  Bob  | Math    | 75    | 3
 Charlie | English | 89    | 1
 Charlie | Math    | 85    | 2
 Charlie | Science | 82    | 3
(9 rows)
```

If your confused take a look at Simple Example of finding popularity of names:

```
postgres=# select * from names;
 gender | name      | total
-----+-----+-----
 boy    | ram       | 100
 girl   | seetha    | 100
 boy    | abdul    | 90
 girl   | tasleem   | 80
 boy    | james     | 70
 girl   | brave     | 70
(6 rows)
```

```

postgres=# select gender,name,total, row_number() over(order by total asc) as popular from names;
gender | name | total | popular
-----+-----+-----+-----
boy    | james | 70    | 1
girl   | brave | 70    | 2
girl   | tasleem | 80    | 3
boy    | abdul | 90    | 4
boy    | ram    | 100   | 5
girl   | seetha | 100   | 6
(6 rows)

postgres=# select gender,name,total, rank() over(order by total asc) as popular
from names;
gender | name | total | popular
-----+-----+-----+-----
boy    | james | 70    | 1
girl   | brave | 70    | 1
girl   | tasleem | 80    | 3
boy    | abdul | 90    | 4
boy    | ram    | 100   | 5
girl   | seetha | 100   | 5
(6 rows)

postgres=# select gender,name,total, dense_rank() over(order by total asc) as po
pular from names;
gender | name | total | popular
-----+-----+-----+-----
boy    | james | 70    | 1
girl   | brave | 70    | 1
girl   | tasleem | 80    | 2
boy    | abdul | 90    | 3
boy    | ram    | 100   | 4
girl   | seetha | 100   | 4
(6 rows)

```

PostgreSQL Transactions

1. What is a Transaction?

A **transaction** in PostgreSQL is a sequence of one or more database operations that are executed as a **single unit**. It ensures **consistency** and **integrity** of the database by following the **ACID properties**.

ACID Properties in Transactions

1. **Atomicity** – All operations in a transaction **must succeed** or **none** of them will be executed.
2. **Consistency** – The database remains in a **valid state** before and after a transaction.
3. **Isolation** – Transactions execute **independently** of each other to prevent data corruption.
4. **Durability** – Once a transaction is committed, it is **permanently stored** even if the server crashes.

2. How to Use Transactions in PostgreSQL?

Basic Syntax

- Transactions in PostgreSQL **start with** BEGIN TRANSACTION and **end with** either COMMIT (to save) or ROLLBACK (to undo).

Example: Using BEGIN, COMMIT, and ROLLBACK

```
BEGIN TRANSACTION;
```

```
DELETE FROM employees WHERE salary < 20000;
```

```
ROLLBACK; -- Cancels the transaction
```

```
SELECT * FROM employees; -- The deleted records are still there
```

Here, the ROLLBACK ensures that the DELETE operation is undone.

3. Real-World Example: Bank Transactions

When transferring money between two accounts, both updates must happen together to **prevent data inconsistency**.

Example: Money Transfer Between Two Accounts

```
BEGIN TRANSACTION;
```

```
UPDATE account SET balance = balance - 5000 WHERE account_id = 1001; -- Deduct money
```

```
UPDATE account SET balance = balance + 5000 WHERE account_id = 2002; -- Add money
```

COMMIT; -- Saves the transaction

- If the **second update fails**, the first update will **not be saved**, preventing loss of money.
- If an error occurs before COMMIT, the transaction **automatically rolls back**.

4. Implicit Transactions in PostgreSQL

- Every command in PostgreSQL is **implicitly** part of a transaction.
- If a command (e.g., DELETE FROM table WHERE condition;) **fails**, PostgreSQL **automatically rolls back** that operation.

5. Handling Failures Using ROLLBACK

- If an error occurs during a transaction, PostgreSQL **undoes** all operations in that transaction.
- ROLLBACK ensures that partial updates are **not saved**.

Example: Rolling Back a Transaction

```
BEGIN TRANSACTION;
```

```
UPDATE products SET stock = stock - 5 WHERE product_id = 101;
```

```
UPDATE orders SET status = 'Completed' WHERE order_id = 202;
```

```
ROLLBACK; -- Undoes both updates
```

- If an error occurs (e.g., product stock goes negative), **all changes are reversed**.

6. Nested Transactions and SAVEPOINT

- PostgreSQL allows creating **sub-transactions** using SAVEPOINT.
- If an error occurs, we can **rollback** to a **specific point** instead of undoing the whole transaction.

Example: Using SAVEPOINT

```
BEGIN TRANSACTION;
```

```
UPDATE customers SET status = 'Gold' WHERE customer_id = 1;
```

```
SAVEPOINT save1; -- Creates a savepoint
```

```
UPDATE customers SET discount = 10 WHERE customer_id = 1;
```

```
ROLLBACK TO save1; -- Undo only the discount update
```

```
COMMIT; -- Saves the transaction
```

- The **customer status update remains**, but the **discount update is undone**.

Conclusion

- **Transactions** ensure **data integrity** in PostgreSQL.
- Always use BEGIN TRANSACTION, COMMIT, and ROLLBACK to control **data changes**.
- Use **SAVEPOINT** for finer control in complex operations.
- PostgreSQL **automatically rolls back** transactions if an error occurs.

Functions in PostgreSQL

A **function** in PostgreSQL is a reusable database object that returns a value or a table after executing SQL statements. Unlike **procedures**, functions **cannot** use COMMIT or ROLLBACK but are useful for calculations, data transformations, and queries.

1. Creating a Function

Basic Syntax

```
CREATE FUNCTION function_name(param1 datatype, param2 datatype)
```

```

RETURNS return_type
LANGUAGE plpgsql
AS $$
BEGIN
    -- SQL Statements
    RETURN value;
END;
$$;

```

2. Example: Simple Function

This function adds two numbers and returns the result.

```

CREATE FUNCTION add_numbers(a INT, b INT)
RETURNS INT
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN a + b;
END;
$$;

```

Calling the Function

```
SELECT add_numbers(5, 10); -- Output: 15
```

3. Example: Function Returning a Table

This function retrieves users above a certain age.

```

CREATE FUNCTION get_users_by_age(min_age INT)
RETURNS TABLE(id INT, name TEXT, age INT)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY SELECT * FROM users WHERE age >= min_age;
END;
$$;

```

Calling the Function

```
SELECT * FROM get_users_by_age(30);
```

4. Example: Function with IF-ELSE Logic

A function to determine if a number is **even** or **odd**.

```

CREATE FUNCTION check_even_odd(n INT)
RETURNS TEXT
LANGUAGE plpgsql
AS $$
BEGIN
    IF n % 2 = 0 THEN
        RETURN 'Even';
    ELSE
        RETURN 'Odd';
    END IF;

```


END;

\$\$;

Calling the Function

SELECT check_even_odd(7); -- Output: Odd

5. Example: Function with an OUT Parameter

CREATE FUNCTION get_balance(acc_id INT, OUT balance DECIMAL)

LANGUAGE plpgsql

AS \$\$

BEGIN

SELECT account_balance INTO balance FROM accounts WHERE id = acc_id;

END;

\$\$;

Calling the Function

SELECT get_balance(101);

Dropping a Function

To remove a function:

DROP FUNCTION add_numbers(INT, INT);

or if it exists:

DROP FUNCTION IF EXISTS add_numbers(INT, INT);

8. Benefits of Functions

Encapsulation: Reuse complex queries as a function.

Performance: Runs inside PostgreSQL, reducing client-server interaction.

Security: Can restrict direct table access and return controlled data.

Stored Procedures in PostgreSQL

A **Stored Procedure** in PostgreSQL is a set of SQL statements that are saved in the database and can be executed as a single unit. Unlike functions, stored procedures allow **transaction control** (i.e., you can use COMMIT and ROLLBACK inside them).

Creating a Stored Procedure

You define a stored procedure using the CREATE PROCEDURE statement.

Basic Syntax

CREATE PROCEDURE procedure_name (param1 datatype, param2 datatype)

LANGUAGE plpgsql

AS \$\$

BEGIN

-- SQL statements

END;

\$\$;

Example 1: A Simple Procedure

This procedure inserts a new record into a users table.

CREATE PROCEDURE add_user(IN username TEXT, IN user_age INT)

```

LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO users (name, age) VALUES (username, user_age);
END;
$$;

```

Calling the Procedure

```
CALL add_user('John Doe', 30);
```

Example 2: Using Transactions Inside a Procedure

```

CREATE PROCEDURE transfer_funds(IN sender_id INT, IN receiver_id INT, IN amount DECIMAL)
LANGUAGE plpgsql
AS $$
BEGIN
    BEGIN;
        UPDATE accounts SET balance = balance - amount WHERE id = sender_id;
        UPDATE accounts SET balance = balance + amount WHERE id = receiver_id;
    COMMIT;
END;
$$;

```

Calling the Procedure

```
CALL transfer_funds(1, 2, 500);
```

Differences Between Stored Procedures & Functions

Feature	Stored Procedure	Function
Returns a Value	No (Use OUT parameters)	Yes
Can Commit/Rollback	Yes	No
Used for	Complex transactions	Computations & Queries
Called with	CALL	SELECT or EXECUTE

Modifying & Dropping a Procedure

Modify a Procedure

To modify, you need to DROP and re-create it.

```
DROP PROCEDURE IF EXISTS add_user;
```

Then, create it again with the modifications.

Drop a Procedure

```
DROP PROCEDURE add_user;
```

create individual stored procedures in PostgreSQL using :

- IN parameters
- OUT parameters
- INOUT parameters

We'll use the same employee table for consistency.

First: Table Setup (if not already done)

```
CREATE TABLE employee (
```

```

emp_id SERIAL PRIMARY KEY,
emp_name TEXT NOT NULL,
department TEXT,
salary NUMERIC
);

```

```

INSERT INTO employee (emp_name, department, salary) VALUES
('Alice', 'HR', 40000),
('Bob', 'Finance', 55000),
('Charlie', 'IT', 60000);

```

1. Procedure with IN Parameters

Takes inputs, **does something**, no output returned.

```

CREATE OR REPLACE PROCEDURE add_employee(
    IN name TEXT,
    IN dept TEXT,
    IN sal NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO employee(emp_name, department, salary)
    VALUES (name, dept, sal);
END;
$$;

```

Call it:

```
CALL add_employee('David', 'Marketing', 45000);
```

2. Procedure with OUT Parameters

No inputs, just **returns data**.

```

CREATE OR REPLACE PROCEDURE get_highest_paid(
    OUT name TEXT,
    OUT max_salary NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    SELECT emp_name, salary INTO name, max_salary
    FROM employee
    ORDER BY salary DESC
    LIMIT 1;
END;
$$;

```

Call it:

```
CALL get_highest_paid(name, max_salary);
```

This will return the name and salary of the highest-paid employee.

3. Procedure with INOUT Parameters

Takes a value, modifies it, and **returns the updated value**.

```

CREATE OR REPLACE PROCEDURE adjust_salary(
    IN empid INT,
    INOUT bonus NUMERIC
)
LANGUAGE plpgsql
AS $$
DECLARE
    sal NUMERIC;
BEGIN
    SELECT salary INTO sal FROM employee WHERE emp_id = empid;

    IF sal > 50000 THEN
        bonus := bonus + 3000;
    ELSE
        bonus := bonus + 1500;
    END IF;
END;
$$;
Call it:
CALL adjust_salary(2, 1000); -- For employee ID 2, bonus becomes 4000

```

Key Benefits of Stored Procedures

Encapsulation: Groups multiple SQL statements.

Performance: Reduces network overhead by executing SQL at the database level.

Security: Can restrict access to sensitive operations.

Transaction Management: Can use COMMIT and ROLLBACK within the procedure.

What Are Triggers in PostgreSQL?

A **trigger** in PostgreSQL is a database mechanism that automatically **executes a specified function** in response to certain **events** (like INSERT, UPDATE, or DELETE) on a table or view.

Why Use Triggers?

- To maintain **audit logs**
- To **validate data** before insert/update
- To **automate actions** on table changes (e.g., update timestamp)
- To **enforce business rules**

Types of Triggers:

Type	Description
BEFORE Trigger	Executes before an operation (INSERT/UPDATE/DELETE)
AFTER Trigger	Executes after an operation
INSTEAD OF Trigger	Used with views (since views don't store data directly)

Trigger Structure:

1. **Trigger Function** (written in PL/pgSQL)
2. **Trigger** itself which connects event to the function

Syntax:

1. Create a Trigger Function

```
CREATE OR REPLACE FUNCTION function_name()  
RETURNS TRIGGER AS $$  
BEGIN  
    -- action  
    RETURN NEW; -- or RETURN OLD for DELETE  
END;  
$$ LANGUAGE plpgsql;
```

2. Create the Trigger

```
CREATE TRIGGER trigger_name  
AFTER INSERT OR UPDATE OR DELETE ON table_name  
FOR EACH ROW  
EXECUTE FUNCTION function_name();
```

Example: Auditing INSERTs on an employees table

➤ Step 1: Create Main Table

```
CREATE TABLE employees (  
    emp_id SERIAL PRIMARY KEY,  
    name TEXT,  
    department TEXT  
);
```

➤ Step 2: Create Audit Table

```
CREATE TABLE audit_log (  
    log_id SERIAL PRIMARY KEY,  
    operation TEXT,  
    emp_name TEXT,  
    log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

➤ Step 3: Create the Trigger Function

```
CREATE OR REPLACE FUNCTION log_insert_trigger()  
RETURNS TRIGGER AS $$  
BEGIN  
    INSERT INTO audit_log(operation, emp_name)  
    VALUES ('INSERT', NEW.name);  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

➤ Step 4: Create the Trigger on employees

```
CREATE TRIGGER after_employee_insert  
AFTER INSERT ON employees  
FOR EACH ROW  
EXECUTE FUNCTION log_insert_trigger();
```

➤Step 5: Test the Trigger

```
INSERT INTO employees(name, department) VALUES ('John Doe', 'IT');
```

```
-- Check the audit log
```

```
SELECT * FROM audit_log;
```

Output:

```
log_id | operation | emp_name | log_time
```

```
1      | INSERT   | John Doe | 2025-04-03 18:10:00
```

Notes:

- NEW: A record containing the **new row** for INSERT/UPDATE.
- OLD: A record containing the **old row** for DELETE.
- FOR EACH ROW: Trigger is fired for **each affected row**.

Setup: Create Tables

```
-- Employee Table
```

```
CREATE TABLE employees (  
    id SERIAL PRIMARY KEY,  
    name TEXT NOT NULL,  
    department TEXT  
);
```

```
-- Audit Log Table
```

```
CREATE TABLE audit_log (  
    log_id SERIAL PRIMARY KEY,  
    operation TEXT,  
    emp_name TEXT,  
    log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

1. BEFORE INSERT Trigger (Validate name)

Trigger Function

```
CREATE OR REPLACE FUNCTION validate_name()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF NEW.name IS NULL OR LENGTH(TRIM(NEW.name)) = 0 THEN  
        RAISE EXCEPTION 'Employee name cannot be empty!';  
    END IF;  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

Trigger Creation

```
CREATE TRIGGER validate_name_before_insert  
BEFORE INSERT ON employees  
FOR EACH ROW  
EXECUTE FUNCTION validate_name();
```

Example Usage

```
-- Valid insert
INSERT INTO employees(name, department) VALUES ('John Doe', 'HR');
```

Output:

```
INSERT 0 1
```

```
-- Invalid insert
```

```
INSERT INTO employees(name, department) VALUES (' ', 'IT');
```

Output:

```
ERROR: Employee name cannot be empty!
```

2. AFTER UPDATE Trigger (Track changes)

Trigger Function

```
CREATE OR REPLACE FUNCTION log_department_change()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO audit_log(operation, emp_name)
    VALUES ('UPDATED: ' || NEW.department, NEW.name);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Trigger Creation

```
CREATE TRIGGER after_department_update
AFTER UPDATE ON employees
FOR EACH ROW
WHEN (OLD.department IS DISTINCT FROM NEW.department)
EXECUTE FUNCTION log_department_change();
```

Example Usage

```
-- Update department
```

```
UPDATE employees SET department = 'Finance' WHERE name = 'John Doe';
```

Output:

```
UPDATE 1
```

```
SELECT * FROM audit_log;
```

Output:

```
log_id | operation      | emp_name | log_time
-----+-----+-----+-----
1 | UPDATED: Finance | John Doe | 2025-04-03 10:25:10
```

3. AFTER DELETE Trigger

Trigger Function

```
CREATE OR REPLACE FUNCTION log_deletion()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO audit_log(operation, emp_name)
    VALUES ('DELETE', OLD.name);
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;
```

Trigger Creation

```
CREATE TRIGGER after_employee_delete
AFTER DELETE ON employees
```

```
FOR EACH ROW
EXECUTE FUNCTION log_deletion();
```

Example Usage

```
DELETE FROM employees WHERE name = 'John Doe';
```

Output:

```
DELETE 1
SELECT * FROM audit_log;
```

Output:

log_id	operation	emp_name	log_time
1	UPDATED: Finance	John Doe	2025-04-03 10:25:10
2	DELETE	John Doe	2025-04-03 10:30:15

4. Trigger on INSERT or UPDATE

Function

```
CREATE OR REPLACE FUNCTION audit_insert_update()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO audit_log(operation, emp_name)
        VALUES ('INSERT', NEW.name);
    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO audit_log(operation, emp_name)
        VALUES ('UPDATE', NEW.name);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Trigger

```
CREATE TRIGGER insert_update_audit
AFTER INSERT OR UPDATE ON employees
FOR EACH ROW
EXECUTE FUNCTION audit_insert_update();
```

Example Usage

```
-- Insert new employee
INSERT INTO employees(name, department) VALUES ('Alice', 'Admin');
```

```
-- Update department
UPDATE employees SET department = 'Tech' WHERE name = 'Alice';
```

Output in audit_log:

log_id	operation	emp_name	log_time
3	INSERT	Alice	2025-04-03 10:32:45
4	UPDATE	Alice	2025-04-03 10:35:10

Use `SELECT * FROM audit_log ORDER BY log_id;` to track all events and operations in sequence.

Full-Text Search:

Full-text search allows you to search **natural language text** (like articles, documents, etc.) for specific **words or phrases**, rather than doing simple string matching.

Why use it?

- It's smarter than LIKE '%word%'
- It handles **stemming** (e.g., "searching" = "search")
- It ignores **stop words** like "the", "is", "a", etc.
- You can use **ranking** to sort by relevance

```
postgres=# CREATE TABLE articles (
postgres(#      id SERIAL PRIMARY KEY,
postgres(#      title TEXT,
postgres(#      content TEXT,
postgres(#      document TSVECTOR
postgres(# );

postgres=# CREATE TABLE articles (
postgres(#      id SERIAL PRIMARY KEY,
postgres(#      title TEXT,
postgres(#      content TEXT,
postgres(#      document TSVECTOR
postgres(# );
ERROR: relation "articles" already exists
postgres=# INSERT INTO articles (title, content) VALUES
postgres=# ('PostgreSQL Overview', 'PostgreSQL is a powerful, open-source object-relational database.'),
postgres=# ('Full Text Search', 'Using PostgreSQL for full text search is fast and efficient.'),
postgres=# ('Search Engines', 'PostgreSQL can be used like a search engine with tsvector. ');
INSERT 0 3
postgres=# UPDATE articles
postgres=# SET document = to_tsvector(title || ' ' || content);
UPDATE 6
postgres=# SELECT * FROM articles
postgres=# WHERE document @@ to_tsquery('PostgreSQL & search');
 id | title | content | document
---+-----+-----+-----
  1 | PostgreSQL Full Text | PostgreSQL provides powerful full text search features. | 'featur':10 'full':2,7 'postgresql':1,4 'power':6 'provid':5 'search':9 'text':3,8
  3 | Search Optimization | Full text search in PostgreSQL can be optimized with GIN indexes. | 'full':3 'gin':12 'index':13 'optim':2,10 'postgresql':7 'search':1,5 'text':4
  5 | Full Text Search | Using PostgreSQL for full text search is fast and efficient. | 'effici':13 'fast':11 'full':1,7 'postgresql':5 'search':3,9 'text':2,8 'use':4
  6 | Search Engines | PostgreSQL can be used like a search engine with tsvector. | 'engin':2,10 'like':7 'postgresql':3 'search':1,9 'tsvector':12 'use':6
(4 rows)
```

What this means:

- @@ checks whether the document **matches** the full-text query.
- 'search & indexing' looks for **both** the words "search" and "indexing" in the text.
- But your result **returned only one row**, even though the word "indexing" **doesn't appear** in the output content.

Field	Value
id	3
title	Search Optimization
content	Full text search in PostgreSQL can be optimized with GIN indexes.
document	'full':3 'gin':12 'index':13 'optim':2,10 'postgresql':7 'search':1,5 'text':4

How to read the document column (TSVECTOR):

This is a **tokenized and stemmed** version of the text.

It means:

- search: appears at positions **1** and **5**
- index: appears at position **13** (note: "indexes" is **stemmed** to "index")
- optim: is the stemmed version of "optimization" and "optimized"
- 'gin':12 — "GIN" appears at word position 12
- 'full':3 — "full" is word #3
- PostgreSQL removes stopwords like "can", "be", "with"

Stemmed Words:

PostgreSQL uses **dictionaries** and **parsers** to stem words:

- "indexes" → index
- "optimization", "optimized" → optim

- This allows **better matching** even if exact words differ slightly

Multidimensional Data :

Definition:

Multidimensional data refers to data that has multiple values for one item — like coordinates (x, y), or more (x, y, z, ...). PostgreSQL supports this using the cube extension.

Why use it?

- Great for **spatial data**, **scientific data**, or anything involving vectors
- Can compare **points**, **ranges**, and **overlaps** in multidimensional space
- Useful for AI, geometry, analytics, etc.

```
postgres=# CREATE EXTENSION IF NOT EXISTS cube;
NOTICE: extension "cube" already exists, skipping
CREATE EXTENSION
postgres=# CREATE TABLE points3d (
postgres=#     id SERIAL PRIMARY KEY,
postgres=#     location CUBE
postgres=# );
CREATE TABLE
postgres=# INSERT INTO points3d (location) VALUES
postgres=#     (cube(ARRAY[1, 2, 3])),
postgres=#     (cube(ARRAY[3, 4, 5])),
postgres=#     (cube(ARRAY[6, 7, 8])),
postgres=#     (cube(ARRAY[2, 3, 4]));
INSERT 0 4
postgres=# SELECT * FROM points3d
postgres=# WHERE location && cube(ARRAY[3, 4, 5]);
 id | location
----+-----
  2 | (3, 4, 5)
(1 row)

postgres=# SELECT * FROM points3d
postgres=# WHERE location <@ cube(ARRAY[1,2,3], ARRAY[4,5,6]);
 id | location
----+-----
  1 | (1, 2, 3)
  2 | (3, 4, 5)
  4 | (2, 3, 4)
(3 rows)
```

- SELECT * FROM points3d: Retrieves all columns (id, location) from the table.
- WHERE location && cube(ARRAY[3, 4, 5]):
 - The && operator checks for **overlap**.
 - cube(ARRAY[3, 4, 5]) creates a 3D point (or a cube of zero volume at this point).
 - This query returns any rows whose location cube **overlaps** with the point (3, 4, 5).
 - In this context, since points have zero volume, it only matches **exact points**.

operator	meaning
&&	Overlaps
<@	Contained within
@>	Contains
~=	Equals
<#>	Distance between cube centers

What is Fuzzy Searching?

Fuzzy searching finds matches **even if the search terms are not exact** – it's useful when:

- You expect **typos or spelling variations**.
- You're matching **user input** to data.
- You want **partial or approximate matches**.

PostgreSQL for Fuzzy Searching

Method	Purpose	Extension Needed
ILIKE, SIMILAR TO	Basic pattern matching	No

pg_trgm (Trigram Matching)	Similarity-based word matching	Yes
Levenshtein()	Edit distance (number of changes)	Yes (fuzzystrmatch)

1. Using ILIKE (case-insensitive LIKE)

- LIKE is used for pattern matching using wildcard characters.
- ILIKE is the case-insensitive version of LIKE

Wildcards:

- %: matches **zero or more characters**.
- _: matches **exactly one character**.

-- Case-sensitive search

```
SELECT * FROM movies WHERE title LIKE 'Star%';
```

-- Case-insensitive search

```
SELECT * FROM movies WHERE title ILIKE 'star%';
```

-- Match any title starting with 'Stardust'

```
SELECT title FROM movies WHERE title ILIKE 'stardust%';
```

-- Match titles starting with 'Stardust' followed by at least one more character

```
SELECT title FROM movies WHERE title ILIKE 'stardust_%';
```

-- Find titles where second letter is 'a'

```
SELECT title FROM movies WHERE title LIKE '_a%';
```

Example:

```
postgres=# INSERT INTO movies (title, genre, year) VALUES
postgres-# ('The Matrix', 'Sci-Fi', 1999),
postgres-# ('The Godfather', 'Crime', 1972),
postgres-# ('The Dark Knight', 'Action', 2008),
postgres-# ('Spider-Man', 'Action', 2002),
postgres-# ('Iron Man', 'Action', 2008),
postgres-# ('Inside Out', 'Animation', 2015),
postgres-# ('Star Wars', 'Sci-Fi', 1977),
postgres-# ('Stardust', 'Fantasy', 2007),
postgres-# ('The Notebook', 'Romance', 2004),
postgres-# ('Gone Girl', 'Thriller', 2014);
INSERT 0 10
postgres=# SELECT * FROM movies WHERE title LIKE 'Star%';
 id | title      | genre | year
----+-----+-----+----
  7 | Star Wars  | Sci-Fi | 1977
  8 | Stardust   | Fantasy | 2007
(2 rows)

postgres=# SELECT * FROM movies WHERE title ILIKE 'star%';
 id | title      | genre | year
----+-----+-----+----
  7 | Star Wars  | Sci-Fi | 1977
  8 | Stardust   | Fantasy | 2007
(2 rows)

postgres=# SELECT title FROM movies WHERE title ILIKE 'stardust%';
 title
-----
 Stardust
(1 row)

postgres=# SELECT title FROM movies WHERE title ILIKE 'stardust_%';
 title
-----
(0 rows)
```

2. Trigram Matching (pg_trgm)

Step 1: Enable Extension

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
```

Step 2: Sample Table

```
CREATE TABLE customers (
  id SERIAL PRIMARY KEY,
  name TEXT
);
```

```
INSERT INTO customers (name) VALUES
```

```

('Jonathan'),
('John'),
('Jon'),
('Johnny'),
('Joan'),
('Janet');

```

```

postgres=# CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE EXTENSION
postgres=# CREATE TABLE customers (
postgres=#         id SERIAL PRIMARY KEY,
postgres=#         name TEXT
postgres=# );
CREATE TABLE
postgres=# INSERT INTO customers (name) VALUES
postgres-# ('Jonathan'),
postgres-# ('John'),
postgres-# ('Jon'),
postgres-# ('Johnny'),
postgres-# ('Joan'),
postgres-# ('Janet');
INSERT 0 6

```

Step 3: Use % operator or similarity()

```

-- Find similar names to 'Jon'
SELECT name, similarity(name, 'Jon') AS score
FROM customers
WHERE similarity(name, 'Jon') > 0.1
ORDER BY score DESC;

```

Similarity is Calculated by using Formula(based on Dice coefficient):

$$\text{similarity}(A, B) = \frac{2 \times (\text{number of shared trigrams})}{(\text{total trigrams of } A + B)}$$

Name	Trigrams (4+ each)	Shared Trigrams with Jon	Similarity Score	Notes
Jon	['j', 'jo', 'jon', 'on ']	4	1.000	Exact match
Jonathan	['j', 'jo', 'jon', 'ona', 'nat', 'ath', 'tha', 'han', 'an ']	3	0.300	Prefix match with "Jon"
John	['j', 'jo', 'joh', 'ohn', 'hn ']	2	0.2857	Starts with "Joh"
Joan	['j', 'jo', 'joa', 'oan', 'an ']	2	0.2857	Similar start and pattern
Johnny	['j', 'jo', 'joh', 'ohn', 'hnn', 'nny', 'ny ']	2	0.2222	Starts with "Joh", longer
Janet	['j', 'ja', 'jan', 'ane', 'net', 'et ']	1	0.1111	Only "j" is common

Output:

```

postgres=# SELECT name, similarity(name, 'Jon') AS score
postgres=# FROM customers
postgres=# WHERE similarity(name, 'Jon') > 0.1
postgres=# ORDER BY score DESC;

```

name	score
Jon	1
Jonathan	0.3
John	0.2857143
Joan	0.2857143
Johnny	0.22222222
Janet	0.11111111

(6 rows)

3. Levenshtein Distance (fuzzystrmatch)

Step 1: Enable Extension

```
CREATE EXTENSION IF NOT EXISTS fuzzystrmatch;
```

Step 2: Use Levenshtein Function

```
SELECT name, levenshtein(name, 'Jon') AS distance
```

```
FROM customers
```

```
ORDER BY distance;
```

Output:

```

postgres=# CREATE EXTENSION IF NOT EXISTS fuzzystrmatch;
postgres=# CREATE EXTENSION
postgres=# SELECT name, levenshtein(name, 'Jon') AS distance
postgres=# FROM customers
postgres=# ORDER BY distance;

```

name	distance
Jon	0
John	1
Joan	1
Johnny	3
Janet	3
Jonathan	5

(6 rows)

Lower value = closer match.

4. Fuzzy Filtering with Index (for performance)

You can create a GIN index for pg_trgm to speed up fuzzy matching.

```
CREATE INDEX trgm_idx ON customers USING GIN (name gin_trgm_ops);
```

Then use % operator:

```
SELECT * FROM customers WHERE name % 'Jon';
```

Real Use Cases

- Autocomplete features
- Search bars (e.g., "Did you mean...")
- Spell-checking systems
- Fuzzy name matching (e.g., voter registration, client databases)

PostgreSQL: Strengths and Weaknesses

Strengths of PostgreSQL

1. **Mature and Robust**
 - Built on decades of research and production usage.
 - Proven reliability across diverse computing fields.
2. **Flexible Querying**
 - Powerful SQL engine supports joins, filters, views, and indexes.
 - Queries can evolve over time without changing the core schema.
3. **Driver & Language Support**
 - Well-supported across all major programming languages.
 - Seamlessly integrates with ORMs(Object-Relational Mapping) and other development tools.
4. **Schema Design**
 - Excellent for structured and uniform (“Stepford”) data.
 - Supports normalization and consistent data modeling.
5. **Advanced Customization**
 - Support for custom data types, functions, indexes, and extensions.
 - Ability to override query parsing or create user-defined languages.
6. **Powerful Constraint Mechanisms**
 - Strong integrity enforcement through constraints like CHECK, UNIQUE, FOREIGN KEY, etc.
7. **True Open Source**
 - Permissive license: no vendor lock-in, no complex legal terms.
 - Fully community-driven with transparent development processes.
8. **ACID Compliance**
 - Supports atomic, consistent, isolated, and durable transactions for strong data integrity.

Weaknesses of PostgreSQL

1. **Partitioning Limitations**
 - Not ideal for large-scale horizontal partitioning (sharding).
 - Better suited for vertical scaling (single powerful machine).
2. **Rigid Schema Requirements**
 - Not great for unstructured or highly dynamic data.
 - May not suit NoSQL-like flexibility or document storage needs.
3. **High-Volume Key-Value Scenarios**
 - Lacks native optimizations for simple, massive key-value read/write workloads.
 - Alternatives like Redis, Cassandra, or DynamoDB may be better here.
4. **Large Binary Data (BLOBs)**
 - PostgreSQL can store blobs, but it’s not optimized for handling very large binary objects at scale.

Conclusion

- PostgreSQL is ideal when **query flexibility** and **data consistency** are essential.
- Schema-first approach provides strong structure with long-term adaptability.
- With proper **normalization, indexing, and tuning**, it can handle **terabytes of data** efficiently.

- For use cases requiring **strong data safety**, **ACID transactions** guarantee reliable and predictable outcomes.

Document Database: Introduction and Its Types

Introduction

A **Document Database** is a type of **NoSQL database** designed to store, retrieve, and manage **semi-structured data** as documents. These documents are typically encoded in **JSON**, **BSON**, or **XML** formats.

Each document contains pairs of **fields (keys)** and **values**, which can be of various data types, including arrays and nested documents. Document databases are schema-less or schema-flexible, making them ideal for applications with evolving or complex data models.

Key Features

- **Schema Flexibility**: No fixed schema — different documents in the same collection can have different fields.
- **Rich Querying**: Supports querying nested fields, full-text search, geospatial queries, and more.
- **High Scalability**: Built for horizontal scaling and distributed architectures.
- **Data as JSON**: Easy to map to object-oriented programming paradigms.

Advantages

- Rapid development and iteration.
- Natural fit for hierarchical or nested data.
- Supports aggregation, indexing, and advanced query operations.

Types of Document Databases

Type	Description
JSON-based	Stores documents in JSON format. Examples: CouchDB , RethinkDB .
BSON-based	Binary-encoded JSON-like storage with additional data types. Examples: MongoDB .
XML-based	Stores and queries documents using XML structure. Examples: BaseX , eXist-db .
Hybrid Document Stores	Combines document and other NoSQL models (e.g., key-value). Example: ArangoDB (multi-model).
Cloud-native Document DBs	Fully managed services offered in cloud platforms. Examples: Amazon DocumentDB , Azure Cosmos DB (Document API), Google Firestore .

Popular Document Databases

1. **MongoDB**
 - Stores data as BSON (Binary JSON).
 - Powerful aggregation framework, rich indexing, and support for ACID transactions.
2. **CouchDB**
 - Uses JSON for documents, JavaScript for MapReduce queries.
 - Designed for high availability and replication.
3. **Amazon DocumentDB**
 - Managed MongoDB-compatible database on AWS.
4. **Firebase Firestore**
 - Real-time NoSQL document database by Google for mobile/web apps.
5. **ArangoDB**
 - Multi-model database supporting documents, graphs, and key-value storage.